A Competitive Programming Field Guide

Game Theory

for Competitive Programming

From First Principles to Contest-Ready Technique

About this guide

What you will learn

Every standard game-theory idea used in competitive programming: positions, P- and N-classification, and the win/lose recursion; Bash-style take-away games; Nim and Bouton's theorem; the misère rule; staircase, Moore's, Fibonacci, and Wythoff's variants; Grundy numbers and the Sprague–Grundy theorem; games on cyclic graphs solved by retrograde analysis; bitmask game dynamic programming; and minimax search with alpha–beta pruning.

Prerequisites

Working knowledge of C++ (recursion, vectors, maps), basic Big–O reasoning, and comfort with induction arguments. Every game-theoretic concept is built from scratch inside; no prior exposure to combinatorial game theory is assumed.

Conventions

All games are two-player with perfect information and, unless stated otherwise, *impartial* under *normal play*: the player who cannot move loses. "Winning position" always means winning for the *player about to move*, with both sides playing optimally. Code is C++17.

October 18, 2026

Contents

1 Introduction **3**

2 Prerequisites: The Anatomy of a Game **3**

2.1 What counts as a game . . . 3
2.2 Game graphs . . . 4
2.3 P-positions and N-positions . . . 4
2.4 A complete worked example . . . 5
2.5 Background assumptions . . . 6

3 The Game-Theoretic Toolbox **6**

3.1 The state-space recipe . . . 6
3.2 The Bash game . . . 7
3.3 State design in action: subtract a digit . . . 7
3.4 Sums of games: where labels run out . . . 8

4 Classic Nim and Its Variants **9**

4.1 Nim and Bouton's theorem . . . 9
4.2 Misère Nim: when the last stone loses . . . 10
4.3 Staircase Nim and the pairing argument . . . 11
4.4 Moore's Nim_k: several piles per turn . . . 11
4.5 Two famous one-and-two-pile games . . . 12
4.6 The standard arguments . . . 12
4.7 The variant shelf at a glance . . . 13

5 Grundy Numbers and the Sprague–Grundy Theorem **13**

5.1 mex and the Grundy value . . . 14
5.2 The Sprague–Grundy theorem . . . 14
5.3 Computing Grundy tables . . . 15
5.4 Sums of games in practice . . . 15
5.5 Moves that split a game . . . 16
5.6 Periodicity: stretching tables to 10^{18} . . . 16

6 Games on Graphs: Cycles and Draws **17**

6.1 Why cycles break the recursion . . . 17
6.2 Retrograde analysis in three rules . . . 17
6.3 Where it is used . . . 18

7 Bitmask Games and Game DP **19**

7.1 The player-to-move pattern over masks . . . 19
7.2 Win/lose over subsets . . . 19
7.3 Score games over subsets . . . 20
7.4 Visited-set games and parity tricks . . . 20
7.5 Practical notes . . . 21

8 Game Tree Search: Minimax and Alpha–Beta **21**

8.1 Minimax on explicit trees . . . 21
8.2 Negamax: one player, one sign . . . 22
8.3 Alpha–beta pruning . . . 22
8.4 Transposition tables . . . 23
8.5 Choosing the tool . . . 23

9 Pitfall Compendium **23**

10 Cheat Sheet and Practice Roadmap **25**

10.1 The one-page cheat sheet . . . 25
10.2 Practice roadmap . . . 26
10.3 Closing advice . . . 27

1 Introduction

Competitive programming loves games. Not the gambling kind—there is no luck to manage—but *combinatorial games*: two players alternating moves on some shared object, stones or coins or graph tokens, each trying to be the one who makes the last move. To a beginner these problems look unmanageable, because the future branching tree of moves seems astronomically large. The central discovery of the field, and the reason this guide exists, is that the branching almost never matters. Positions fall into two clean classes (winning and losing) that a simple recursion separates, and for the most famous games the recursion *collapses to a formula*: a modulo for one-pile take-away games, a single XOR for Nim, a mex-of-XOR machine for every game that decomposes into independent parts. A problem statement with 10^{18} stones often has a one-line answer.

This document is the game-theory companion to the bit-manipulation volume of this series, and it is organized the same way: foundations first, standard results second, machinery third. We start by defining games precisely and proving the win/lose recursion that everything else stands on. We then tour the classical one-pile and multi-pile results—Bash game, Nim and its misère twin, staircase, Moore's, Fibonacci, and Wythoff's variants—because they supply both the formulas and the proof styles (pairing, mirroring, strategy stealing) that editorials lean on without explanation. The middle of the guide is the Sprague–Grundy theorem, the single most important idea in contest game theory: it reduces every sum of independent games to Nim. The closing chapters are engineering: what to do when the game graph has cycles (retrograde analysis), when states are subsets (bitmask game DP), and when the tree is small enough to search (minimax with alpha–beta pruning).

The payoff is concrete. CSES has dedicated *Nim Game I* and *Nim Game II* in its introductory set; the LeetCode stone-game family (roughly a dozen problems) is a bitmask-DP and interval-DP playground; Codeforces puts "games"-tagged tasks into Div. 2 B/C slots every month, and classics like *Berzerk* (retrograde BFS) and *Interesting Game* (Grundy) are standard fare in educational rounds. AtCoder's beginner contests regularly close with a parity or Nim-flavored task. If you plan to work through those, the roadmap in Section 10 lists curated problems matched to each section of this document.

How to read this guide. Sections 2 and 3 are the foundations: the definitions, the win/lose recursion, and the state-space recipe. Read them in order; nothing later works without them. Sections 4 and 5 are the heart of the standard toolbox—Nim and the Sprague–Grundy theory—and deserve careful, complete reading. Sections 6–8 cover the "big machinery" (graph games with draws, bitmask game DP, game-tree search); they can be read independently, but each reuses the position classes of Section 2. Section 9 is a consolidated gallery of traps worth revisiting before every contest, and Section 10 closes with a cheat sheet and a practice roadmap.

2 Prerequisites: The Anatomy of a Game

Everything in contest game theory rests on one piece of background: a precise vocabulary for what a "position" is and a recursive rule that decides who wins from it. This section builds that vocabulary from scratch. The definitions are short, but they are load-bearing: every technique later in this guide—from Bouton's theorem to retrograde analysis—is either an instance of the recursion below or a shortcut that replaces it. Read the definitions actively and re-derive the toy example; the entire subject is a set of consequences of Definition 2.1 plus induction.

2.1 What counts as a game

The problems judges ask about are almost always of one specific kind, and it is worth stating the boundaries exactly.

Definition 2.1 (Combinatorial game)**.** A *combinatorial game* is a set of *positions* together with a *move relation*: from each position p, a fixed set of positions reachable in one move. Two players alternate moves, one at a time, starting from an initial position. Both players see everything (*perfect information*), chance plays no role, and play is *finite*: every sequence of moves ends after finitely many moves. The game ends when the player to move has no legal move.

Two refinements complete the picture. A game is *impartial* if from any position both players have exactly the same legal moves—stones in piles, tokens on graphs, coins in rows are impartial; chess and checkers are not, because the two sides own different pieces, making the game *partisan*. Under *normal play*, the player who cannot move loses and the opponent is declared the winner; under *misère play* the convention is flipped, and the player who makes the last move loses instead. This guide works under normal play unless a subsection says otherwise, because the standard theory (and the standard problem set) is built for it; the misère versions that appear in contests are handled in Sections 4 and 9.

Two examples fix the intuition. In the *one-pile take-away game* with parameter m, a position is a number of stones n, and a move removes between 1 and m stones; the game ends at $n = 0$. In *Nim* the position is a tuple of pile sizes $(p_1, \ldots, p_k)$, and a move chooses one pile and removes a positive number of stones from it. Both are impartial and finite (each move strictly decreases the total number of stones), which is exactly what the recursion of Section 2.3 needs.

2.2 Game graphs

The cleanest way to reason about any game is to forget the story and keep the structure.

Definition 2.2 (Game graph)**.** The *game graph* of a combinatorial game has one vertex per position and one directed edge $p \to q$ for every legal move from p to q. Positions with no outgoing edges are *terminal*; under normal play they are exactly the positions where the player to move loses.

For Nim the vertices are tuples of non-negative integers, and an edge decreases exactly one coordinate. For a token game on a graph, the vertices are (current vertex) or (current vertex, visited set) pairs—note that the "state" of a game is whatever the *rules* say matters, not whatever the story describes; designing that state is half of every game problem, and Section 3 turns it into a checklist. Finiteness of the game means the game graph is *acyclic*—a DAG—so its vertices can be processed in topological order from the terminals upward. Games whose graphs contain cycles (tokens that may move back to earlier vertices) are genuinely different: recursion on them loops forever, and they get their own algorithm in Section 6.

2.3 P-positions and N-positions

The entire theory is a two-label classification of positions. The names are historical but precise: an *N-position* is a win for the *N*ext player (the one about to move), and a *P-position* is a win for the *P*revious player—equivalently, a loss for the player about to move, assuming optimal play on both sides.

Theorem 2.1 (The fundamental classification)**.** Consider a finite impartial game under normal play. Then every position is exactly one of the two classes below, and they are characterized recursively:

- a position with no legal moves (a terminal) is a **P-position**;
- a position from which *some* move leads to a P-position is an **N-position**;
- a position from which *every* move leads to an N-position is a **P-position**.

Proof. Since the game graph is a DAG, classify each vertex by induction on the longest play starting from it (terminals have length zero). A terminal is a P-position by the normal-play

rule, and the two non-terminal clauses are mutually exclusive and exhaustive: either some move lands on a P-position or none does. This assigns every position exactly one label. It remains to check that the labels describe optimal play. From an N-position the mover plays to the P-position that exists by definition; from then on, whatever the opponent does returns play to an N-position (every move out of a P-position lands on an N-position), so the first player can repeat the strategy after each reply. Because every play is finite and terminals are P-positions, the last position reached before the opponent is stuck is a P-position reached *by* the first player—the opponent is the one unable to move. The dual argument shows the second player wins from a P-position: every first-player move hands over an N-position, from which the second player again moves to a P-position, and finiteness ends the play with the first player stuck. □

Two remarks before we turn the theorem into code. First, the theorem says nothing about *how many* winning moves exist or how large they are—those are extras that individual games supply (Nim supplies them, Section 4). Second, the recursion is exactly a topological evaluation over the game graph, which is why "solve the game" and "run a DP over the state graph" are the same sentence. That observation is the entire content of the next subsection.

```
// Subtraction game {1,2,3}: remove 1..3 stones; unable to move = lose.
// memo[u] == -1 unknown, 0 = P-position (mover loses), 1 = N-position.
vector<char> memo(N + 1, -1);

bool wins(int n) {                      // perspective: player to move
    if (n == 0) return false;           // terminal -> P-position
    char &m = memo[n];
    if (m != -1) return m;              // already classified
    bool w = false;
    for (int take : {1, 2, 3})          // enumerate legal moves
        if (!wins(n - take)) { w = true; break; }  // found move to P
    return m = w;                       // N iff some move reaches P
}
```

Listing 1: The fundamental recursion as code: win/lose DP from the mover's perspective.

The listing encodes the convention that keeps every game DP in this guide simple: `wins(state)` always answers *for the player about to move from state*. Opponent turns are never threaded through the recursion as a parameter; instead, "my win" is defined as "some move of mine leads to a state that is a loss *for the next mover*", which is the same function. This idiom removes an entire class of turn-tracking bugs, and Section 9 returns to it as a checklist item.

2.4 A complete worked example

Apply Theorem 2.1 by hand to the subtraction game $\{1, 2, 3\}$—take between one and three stones, last stone wins. The terminal $n = 0$ is a P-position. Positions $1, 2, 3$ can move to 0, so each is an N-position. Position 4 has moves only to $3, 2, 1$, all N-positions, so 4 is a P-position; and the pattern repeats with period 4: from $n = 4q + r$ with $0 < r \leq 3$ the mover takes r stones and hands over the P-position $4q$. Table 1 shows the classification; the periodicity it reveals is the baby version of the Bash-game formula of Section 3.2 and of the period-finding recipe of Section 5.6.

Table 1: Win/lose classes for the subtraction game $\{1, 2, 3\}$. P means the player *to move* loses with optimal play.

position n	0	1	2	3	4	5	6	7	8
class	P	N	N	N	P	N	N	N	P
winning move	–	-1	-2	-3	–	-1	-2	-3	–

Notice what the table teaches about *strategy statements*. "Win by taking r stones" is a complete, checkable description of the winning move, and it was found by evaluating eight positions, not by simulating games. This is the shape of nearly every game problem you will meet: the recursion is cheap on a small range, the pattern is extracted, and the formula is *proved* by checking the two clauses of Theorem 2.1 (from an N-position exhibit a move to a P-position; from a P-position show every move lands on an N-position). The pattern-then-proof habit is formalized as a recipe in Section 3.

2.5 Background assumptions

The rest of the guide assumes only standard first-year tools, and it is worth listing them so you can patch any gap deliberately. *Recursion and memoization*: every solver in this guide is a memoized recursion over states, so fluency with `vector`-based memo tables and with converting recursion to iteration is assumed. *Graph basics*: the game graph is a directed graph, and Section 6 runs a reverse BFS over it, so directed adjacency lists, BFS queues, and in-degree bookkeeping should feel routine. *Induction and invariant arguments*: proofs here are two-line inductions over the game graph or parity/invariant counts; nothing deeper. *Big–O over state spaces*: the cost of a game DP is (number of states) $\times$ (cost of enumerating moves from a state), and recognizing when that product fits in the limits—for example $2^n \cdot n$ for subset states with $n \leq 22$—is the main feasibility skill. Finally, a little *XOR fluency* helps: Nim and Grundy theory run on XOR, and the companion bit-manipulation volume covers everything needed; the identity $a \oplus a = 0$ and the fact that XOR of a set is 0 iff every element cancels in pairs are the only facts used repeatedly here.

3 The Game-Theoretic Toolbox

Theorem 2.1 is an algorithm, but an unaffordable one on the positions judges actually ask about: 10^{18} stones cannot be enumerated. Real game problems are solved by a loop that this section makes explicit: *design the state*, *run the recursion on a small window*, *spot the pattern*, *prove the pattern satisfies the two clauses*, *answer in* $O(1)$. We drill the loop on the single most reused result—the Bash game—and then on a state-design example where the interesting work is choosing what the state even is. The section closes by showing where the loop *cannot* help: games that split into independent parts, which need the Grundy theory of Section 5 rather than plain win/lose labels.

3.1 The state-space recipe

Every contest game problem, whatever its story, is answered by the same four questions in the same order.

Technique 3.1 (The four-question recipe)**.**

1. **State:** what facts about the present summarize all that matters for the future? Encode exactly those; anything not in the state must provably never affect the outcome.
2. **Moves:** enumerate the transitions of one turn. If the move set is infinite or huge, look for structure (only the best move can matter, or moves factor through a small set of options).
3. **Base:** which states are terminal, and who wins them under the stated convention (normal or misère play)?
4. **Budget:** how many states are there? If the count exceeds the limits, the intended solution is a *pattern*: run the recursion on the first few dozen states, guess, and prove.

Two habits make the recipe reliable. First, always compute from the *player-to-move* perspective (the convention of Listing 1); mixing perspectives mid-solution is the top source of wrong answers in game problems. Second, when the pattern guess is "periodic with period p" or "P-positions are multiples of q", the proof obligation is never statistics—it is the two clauses

of Theorem 2.1, checked symbolically. A guessed formula that survives the two clauses is a theorem; one that does not was noise. The Bash game below is the cleanest demonstration of the full loop, so we do it slowly once.

3.2 The Bash game

Definition 3.1 (Bash game)**.** One pile of n stones; players alternately remove between 1 and m stones ($m \geq 1$ fixed). Under normal play—the player who takes the last stone wins—the player to move from n loses exactly when $n \equiv 0 \pmod{m+1}$.

The subtraction game $\{1, 2, 3\}$ of Table 1 is the case $m = 3$, with P-positions at multiples of 4. The general claim has a two-line proof that is worth internalizing verbatim, because the same two clauses are re-checked in every formula proof of this guide.

Theorem 3.1 (Bash)**.** In the Bash game with parameter m, the P-positions are exactly the multiples of $m + 1$.

Proof. Let $n = q(m+1)$ be a multiple. Any legal move removes $r \in [1, m]$ stones, leaving $q(m+1) - r \equiv m+1-r \equiv 1, \ldots, m \pmod{m+1}$, i.e. a non-multiple: every move from a multiple lands on a non-multiple. Conversely, from a non-multiple $n = q(m+1) + r$ with $1 \leq r \leq m$, removing exactly r stones lands on the multiple $q(m+1)$. So the two clauses of Theorem 2.1 hold with P-position set "multiples of $m+1$", and since the game is finite, the classification is exactly that set. □

Technique 3.2 (Complement to $m+1$)**.** The Bash strategy has a memorable operational form: keep the pile at a multiple of $m+1$ after *your* move. Equivalently, answer every opponent move of r stones with $m + 1 - r$. This *complement-pairing* viewpoint—respond to each enemy action with its fixed complement—reappears as the pairing and copycat arguments of Section 4.6, and it is the reason the magic constant is $m + 1$ and not m: each round of optimal play consumes exactly $m + 1$ stones.

```cpp
// n stones, take 1..m, last stone wins.  Returns "first player wins".
bool bashWin(long long n, long long m) { return n % (m + 1) != 0; }

// trust, but verify: DP on a window must agree with the formula
bool bashDP(int n, int m) {
    vector<char> w(n + 1, false);      // w[0] = false: mover loses
    for (int v = 1; v <= n; ++v)
        for (int r = 1; r <= m && r <= v && !w[v]; ++r)
            w[v] = !w[v - r];
    return w[n];
}
```

Listing 2: Bash game: the formula and the sanity-check simulator side by side.

The second function in Listing 2 looks like a testing detail, but it is a *method*: whenever you believe you have a game formula, implement both the formula and a small brute-force DP, and let a loop compare them over a window of a few hundred states. Every claim in this guide was and should be checked this way, and Section 9 elevates the habit to a pre-submission checklist item.

3.3 State design in action: subtract a digit

The recipe earns its keep when the state is not handed to you. Here is a standard problem in that style; we solve it completely, because the way the formula emerges is the way formulas emerge everywhere.

Example 3.1 (Subtract a digit)**.** Start with the number n. Players alternately replace the current number v by $v - d$, where d is one of the *decimal digits* of v (a zero digit is not a legal choice). The player who reaches exactly 0 wins—equivalently, the player unable to move loses, and the only position with no moves is $v = 0$ (from any $v > 0$ some nonzero digit exists). Who wins from n?

Question one: the state is the number v itself—the story adds nothing else. Question two: the moves from v are $v - d$ over the nonzero digits d of v, at most nine of them. Question three: the terminal 0 is a P-position. Question four: the full table is huge for n up to 10^{18}, so compute the first few dozen classes and look. The table below lists the P-positions found by running Listing 1 adapted to digit moves:

v	0	10	20	30	40	...
class	P	P	P	P	P	...

The pattern: P-positions are exactly the multiples of 10. Both clauses of Theorem 2.1 are checked in two sentences. From a multiple $v = 10q$, subtracting any nonzero digit d of v gives $10q - d \equiv 10 - d \pmod{10}$, which is a non-multiple because $1 \le d \le 9$: every move leaves a non-multiple. From a non-multiple $v = 10q + r$ with $1 \le r \le 9$, the residue digit r is precisely the *last* decimal digit of v, hence a legal choice, and $v - r = 10q$ lands on the multiple just below: some move reaches a P-position. The formula is proved, and the answer for any 64-bit n is a single modulo.

```
vector<char> memo(N + 1, -1);          // -1 unknown, 0 P, 1 N
bool winsDigit(int v) {
    if (v == 0) return false;
    char &m = memo[v];
    if (m != -1) return m;
    bool w = false;
    int t = v;
    while (t > 0 && !w) {              // enumerate nonzero digits
        int d = t % 10;
        t /= 10;
        if (d > 0 && !winsDigit(v - d)) w = true;
    }
    return m = w;
}
// loop v = 1..200: P-positions found at 0,10,20,... -> conjecture,
// then prove the two clauses (see text). Answer for 64-bit n:
bool firstWins(long long n) { return n % 10 != 0; }
```

Listing 3: Subtract-a-digit: brute-force DP used to *discover* the multiples-of-10 pattern.

The example also shows the recipe's honest limitation: the discovery window and the proof are separate acts, and the proof needed one subtle observation (a non-multiple always *ends* in a nonzero digit that reaches the multiple below it). Expect exactly this texture on real problems—a mechanical DP, a guess, and one clever sentence that makes the guess a theorem.

3.4 Sums of games: where labels run out

Many contest games are played on *several independent boards at once*: several piles (Nim), several chocolate bars, several token tracks. Players alternate, and on each turn a player makes a move in *exactly one* component; the player unable to move—because every component is terminal—loses. This is the *disjoint sum* $G_1 + G_2 + \cdots + G_k$ of the component games.

Plain P/N labels do not compose: two components, each an N-position, can sum to either class depending on structure, so there is no rule of the form "N plus N is anything". Try it on

two Bash piles with $m = 3$: piles $(1, 1)$ are individually N-positions, and indeed the sum is an N-position; piles $(2, 2)$ are individually N-positions too, but the sum is a P-position (each reply mirrors the move in the other pile). Any label-only arithmetic fails, because win/lose loses the information about *how long* a position can be held—which is precisely what a player steering two boards needs to know.

The fix, developed in Section 5, is to replace each component's label by a *number* that measures exactly that holding power: the Grundy value, a non-negative integer that makes $G_1 + \cdots + G_k$ behave like Nim with piles of sizes $g(G_1), \ldots, g(G_k)$. The result—the Sprague–Grundy theorem—is the backbone of contest game theory, and every hour spent on Sections 4 and 5 is an hour spent on making that sentence computational. Before climbing there, we finish the classical formula shelf: the Nim family, where the holding-power numbers turn out to be plain XOR.

4 Classic Nim and Its Variants

Nim is the hydrogen atom of combinatorial games: the simplest multi-pile game, the home of the XOR formula, and the reference point that every general theorem measures itself against. This section proves Bouton's theorem completely, derives the misère rule with an argument clean enough to reproduce in an interview, and then walks the standard variant shelf—staircase, Moore's, Fibonacci, and Wythoff's games—because contest problems are so often one of these wearing a costume. The section closes with the three classical *arguments* (copycat, strategy stealing, invariants) that solve games no formula covers.

4.1 Nim and Bouton's theorem

Definition 4.1 (Nim)**.** A position is a tuple of piles $(p_1, p_2, \ldots, p_k)$ of stones. A move chooses one pile and removes a positive number of stones from it. Under normal play, the player who cannot move (all piles empty) loses.

Small cases hint at structure. One pile is Bash with $m = p_1$: always a win for the mover. Two equal piles (q, q) are a P-position—every reply mirrors—and two unequal piles are an N-position. Three piles already refuse naive analysis: $(1, 2, 3)$ has no mirror structure, yet it is a P-position, while $(1, 2, 4)$ is not. The invariant that orders all of this is the bitwise XOR of the pile sizes, historically called the *nim-sum*:

$$s \;=\; p_1 \oplus p_2 \oplus \cdots \oplus p_k,$$

where $\oplus$ is addition of binary digits without carrying. The nim-sum is a column-wise parity census: bit b of s is the parity of the number of piles having bit b set.

Theorem 4.1 (Bouton, 1901)**.** A Nim position is a P-position if and only if its nim-sum is zero.

Proof. Write s for the nim-sum. We check the two clauses of Theorem 2.1 against the set $Z = \{\text{positions with } s = 0\}$, then anchor the recursion at the terminal.

From $s \neq 0$ some move reaches Z. Let h be the highest set bit of s. Column h of the census is odd, so an odd number of piles has bit h set; pick one, say p. Replacing p by $p' = p \oplus s$ clears bit h (both have it set) and changes only bits below h, so $p' < p$ is a legal reduction. The nim-sum changes from s to $s \oplus p \oplus p' = s \oplus s = 0$: the new position lies in Z.

From $s = 0$ every move leaves Z. A move replaces one pile p by some $p' < p$, so $p \neq p'$ and the new nim-sum is $s \oplus p \oplus p' = p \oplus p' \neq 0$.

Base. The terminal position, all piles zero, has $s = 0$ and is a loss for the mover. Since Nim is finite (the total strictly decreases), induction over the game graph confirms: P-positions are exactly Z. □

The proof doubles as the strategy. The winning player repeatedly moves to $s = 0$; the opponent must leave $s \neq 0$, and the winner returns. Each winning move is computable in one pass, which makes "output the winning move" problems (a classic genre) almost free:

```cpp
// Normal-play Nim. Returns (pile index, stones to remove),
// or (-1, 0) when the player to move loses (nim-sum zero).
pair<int, long long> nimMove(const vector<long long>& a) {
    long long s = 0;
    for (long long p : a) s ^= p;
    if (s == 0) return {-1, 0};          // P-position: mover loses

    int h = 63 - __builtin_clzll(s);     // highest set bit of s
    for (size_t i = 0; i < a.size(); ++i)
        if ((a[i] >> h) & 1) {           // some pile must have it:
            long long to = a[i] ^ s;     // an odd number of them do
            return {(int)i, a[i] - to};  // to < a[i]: legal move,
        }                                // new nim-sum s^a[i]^to = 0
    return {-1, 0};                      // unreachable
}
```

Listing 4: Nim: decide the winner and extract a winning move in one pass.

Example 4.1. Piles $(3, 4, 5)$: $s = 3 \oplus 4 \oplus 5 = 2 \neq 0$, so the mover wins. The highest set bit of $s = 10_2$ is bit 1; piles 3 and 5 have it set. Reducing 3 to $3 \oplus 2 = 1$ gives $(1, 4, 5)$ with nim-sum $1 \oplus 4 \oplus 5 = 0$. Hand the move to your opponent there, and every reply returns the nim-sum to nonzero, where the winning player again has a move back. Total work: one XOR plus one scan.

4.2 Misère Nim: when the last stone loses

Flip one sentence in the rules—the player who takes the *last* stone *loses*—and Nim's analysis bends but does not break. The rule discovered by Bouton in the same paper is one of the most quoted results in the field:

Theorem 4.2 (Misère Nim)**.** Under misère play, let c be the number of piles of size exactly one. If *every* pile has size at most one, the mover loses exactly when c is odd (and also when $c = 0$, the empty position). Otherwise—some pile has size at least two—the mover loses exactly when the nim-sum is zero, exactly as in normal play.

Proof sketch. When all piles are ≤ 1 the play is forced (remove one stone at a time), and the taker of the last stone loses; counting the forced sequence gives the parity rule directly. When some pile is ≥ 2, one observation does the work: *a position with exactly one pile of size* ≥ 2 *never has nim-sum zero*—that lone pile contributes a set bit (position ≥ 1) in the column of its highest bit that no pile of size one can cancel, so its column census is odd. Hence any $s = 0$ position with a pile ≥ 2 has *at least two* big piles, and no single move can consume both: the endgame "everything becomes ones" cannot be entered *by the responder* from $s = 0$. From $s = 0$, max ≥ 2: every move either keeps a big pile and lands on $s \neq 0$ (bad for the mover, by induction), or ... cannot happen by the observation. From $s \neq 0$, max ≥ 2: if at least two piles are big, Bouton's move to $s = 0$ keeps the other big pile, so it is legal and safe by induction; if exactly one pile is big, reduce it directly to 0 or 1, choosing whichever leaves an *odd* count of ones—a forced-loss position for the opponent by the parity rule. Both clauses hold; the terminal anchor is the empty position, a loss for the mover under both play conventions. □

Operationally: *play normal Nim, but on the move that would finish the game, leave an odd number of single-stone piles.* The condition check must come *first*—the parity branch and the XOR branch disagree, and applying Bouton blindly in the all-ones endgame is the classic way to fail a misère task.

```cpp
// Misere play: taking the LAST stone loses. Player to move wins?
bool misereNimWin(const vector<long long>& a) {
    int ones = 0;  bool big = false;
    for (long long p : a) {
        if (p >= 2) big = true;
        ones += (p == 1);
    }
    if (!big)                              // forced one-by-one play:
        return ones % 2 == 0 && ones > 0;  // mover wins iff even & >0
    long long s = 0;                       // some pile >= 2: Bouton
    for (long long p : a) s ^= p;
    return s != 0;
}
```

Listing 5: Misère Nim decision: check the all-ones branch before touching the nim-sum.

4.3 Staircase Nim and the pairing argument

Line up steps $1, 2, \ldots, n$ with a_i coins on step i; step 0 is the floor. A move takes any positive number of coins from some step $i \geq 1$ and moves them one step down; coins reaching the floor leave the game. The player who cannot move (all steps empty) loses. Only half the board matters:

Technique 4.1 (Staircase Nim)**.** Staircase Nim is Nim in disguise on the *odd-numbered steps*: the mover loses exactly when

$$a_1 \oplus a_3 \oplus a_5 \oplus \cdots = 0.$$

Proof sketch. Group each odd step $2j+1$ with the even step $2j+2$ above it as a Nim pile of size a_{2j+1}. If the odd-step XOR is nonzero, the mover applies Bouton's move inside some odd pile (reducing a_{2j+1} by shifting coins to step $2j$, which leaves the odd system). If the XOR is zero, split by the opponent's move type: a move from an *odd* step changes an odd pile and breaks the zero, and the standard Nim reply on odd piles restores it; a move from an *even* step $2j$ merely adds r coins to the odd pile $2j-1$ below it, and the responder immediately moves those same r coins from $2j-1$ down to the floor—both moves cancel, restoring the zero. Play is finite, so the mirroring player makes the last move. □

The proof is worth memorizing in its own right: it is the canonical *pairing argument*, where the second player answers every move with a forced complement and the state variable (odd-step XOR) never leaves its target set. Pairing arguments reappear in Section 4.6 as a general-purpose tool and in the Bash complement strategy of Technique 3.2.

4.4 Moore's Nim$_k$: several piles per turn

Relax Nim's "exactly one pile" to "*at most* k piles, each reduced by a positive amount, in one turn." Moore (1910) found the P-positions: write the pile counts in binary and census every column; the mover loses exactly when *every column sum is divisible by* $k+1$. For $k = 1$ this is Bouton (each column even); for k large enough to empty everything, the all-terminal column census also fits. The winning strategy modifies Bouton's proof to $(k+1)$-adic census balancing, and the decision procedure is a loop over bits:

```cpp
// At most k piles may be reduced per turn. Is the mover losing?
bool mooreLose(const vector<long long>& a, int k) {
    int cnt[64] = {0};
    for (long long p : a)
        while (p) { ++cnt[__builtin_ctzll(p)]; p &= p - 1; }
    for (int b = 0; b < 64; ++b)
```

```
        if (cnt[b] % (k + 1) != 0) return false;
    return true;                        // every column divisible by k+1
}
```

Listing 6: Moore's Nim_k: P-position test by $(k{+}1)$-divisible column sums.

4.5 Two famous one-and-two-pile games

Fibonacci Nim. One pile of n stones; the first move must take between 1 and $n-1$ stones, and every later move may take between 1 and *twice* the previous move. The classical result (via Zeckendorf's theorem, every n a unique sum of non-consecutive Fibonacci numbers): the mover loses exactly when n is a Fibonacci number, and otherwise the winning move is the *largest summand* of n's Zeckendorf representation. The strategy sustains itself because after taking that summand f, the opponent may take at most $2f$, strictly less than the next Fibonacci summand $f' \geq f + f_{\text{prev}}$—so the winner repeats the argument on the remainder. The full proof needs the Zeckendorf machinery; contest use is the statement plus the observation that the Fibonacci threshold $2\times$ previous is what makes the decomposition stable.

Wythoff's game. Two piles (a, b); a move either reduces one pile by a positive amount, or reduces *both* by the same positive amount; the player reaching $(0, 0)$ wins. The diagonal move defeats plain Nim analysis, but the P-positions are again a formula—the *cold positions*

$$(\lfloor n\varphi \rfloor,\ \lfloor n\varphi \rfloor + n), \qquad n = 0, 1, 2, \ldots, \qquad \varphi = \tfrac{1+\sqrt{5}}{2},$$

the pairs $(0, 0)$, $(1, 2)$, $(3, 5)$, $(4, 7)$, $(6, 10)$, and so on. Beatty's theorem—the sequences $\lfloor n\varphi \rfloor$ and $\lfloor n\varphi^2 \rfloor = \lfloor n\varphi \rfloor + n$ partition the positive integers—guarantees these pairs cover every value exactly once, which is precisely what a complete catalog of P-positions requires: from a cold position, every move repeats a coordinate or breaks the gap $b - a$, landing in a covered warm position; from a warm position, either the gap is zero (equalize via the diagonal), or the gap d matches the cold pair $(\lfloor d\varphi \rfloor, \lfloor d\varphi \rfloor + d)$ reachable by shrinking one pile. The contest implementation is three lines, with one floating-point caveat:

```
bool wythoffLose(long long a, long long b) {   // mover loses?
    if (a > b) swap(a, b);
    long long d = b - a;                       // cold gap: unique d
    const long double phi = (1.0L + sqrtl(5.0L)) / 2.0L;
    long long lo = (long long)floorl((long double)d * phi);
    return a == lo;                            // and b == lo + d
}
```

Listing 7: Wythoff's game: cold-position test via the golden ratio (see the precision warning).

4.6 The standard arguments

Formulas cover the named games; *arguments* cover everything else. Three classical patterns solve the majority of unnamed games in contests, and each deserves a slot in your toolkit beside Theorem 2.1.

Technique 4.2 (Copycat / pairing)**.** If the position is symmetric under a fixed-point-free involution (twin piles, a rectangular board's 180-degree rotation, mirrored rows), and every legal move has a legal mirrored reply, then the *second* player wins: reply to each move with its mirror, restoring symmetry after every round. The argument needs no computation, only a symmetry check; it proves (q, q) is a P-position in Nim, solves the classic "place coins on a rectangular table, last to place wins" (mirror through the center), and powers the staircase proof above. When symmetry is *almost* present but broken by one irregular component, analyze that component alone and mirror the rest—this is also how first-player wins are exhibited in symmetric constructions.

Technique 4.3 (Strategy stealing)**.** Some games are provably first-player wins without any constructive strategy. In *Chomp* (a rectangular chocolate bar; a move eats a chosen cell and everything right and below it; the poisoned top-left cell loses), suppose the second player had a winning reply to the opening move that eats only the bottom-right cell. If that reply eats cells beyond the bottom-right one, the first player could have opened with *that* move instead, since the bottom-right cell is irrelevant once gone; if the reply is exactly the bottom-right cell, the first player opens with it and steals it anyway. Contradiction either way: the first player wins—though the argument exhibits no strategy, which is why Chomp on large boards is a famous open computational problem.

Technique 4.4 (Invariants and coloring)**.** Attach a quantity (a color, a parity, a residue) to every position that each move changes in a controlled way, and read the game off it. Prototype: a rook starts at the lower-right of an $n \times m$ board and moves any number of squares left or down; the player who moves it to the upper-left corner wins. Distance to the corner is $(n-1)+(m-1)$, a move decreases one summand—this is two-pile Nim, so the mover loses iff $(n-1) \oplus (m-1) = 0$. Restrict moves to *one* square and the invariant collapses to a parity: the rook alternates square colors, so the mover loses exactly when start and goal share a color. The lesson generalizes: when a game's moves each consume "one unit" of some resource, look for the coloring or parity of that resource first.

4.7 The variant shelf at a glance

Table 2 compresses the section into the reference form you want in contest: given a story about piles, steps, or bars, find the row and answer in one line. Every row's condition is proved by the same two-clause pattern as Theorem 4.1—the repetition is the point.

Table 2: Classical take-away games and their P-position conditions (P = the player *to move* loses with optimal play).

Game	Move	Mover loses iff
Bash (m cap)	take $1..m$ from one pile	$n \equiv 0 \pmod{m+1}$
Nim	reduce any one pile	nim-sum $= 0$
Misère Nim	reduce one pile; last taker loses	all piles ≤ 1: count of ones odd; else nim-sum $= 0$
Staircase	move coins one step down	$a_1 \oplus a_3 \oplus a_5 \oplus \cdots = 0$
Moore's Nim_k	reduce $\le k$ piles	every binary column sum $\equiv 0 \pmod{k+1}$
Fibonacci Nim	next take $\le 2\times$ previous	n is a Fibonacci number
Wythoff	shrink one pile or both equally	(a, b) is a cold pair $(\lfloor n\varphi \rfloor, \lfloor n\varphi \rfloor + n)$

5 Grundy Numbers and the Sprague–Grundy Theorem

Section 3.4 left a promise on the table: positions need *numbers*, not labels, before independent games can be added. This section delivers. The Grundy value of a position measures exactly its "Nim-equivalence"—how big a pile of stones it behaves like—and the Sprague–Grundy theorem states that a sum of independent games is Nim with one pile per component. Every "multi-board" problem on every judge is an application. The theory also pays for itself on single-board problems: many games with no closed formula still have computable Grundy tables, and the tables often turn out periodic, which extends them to 10^{18}.

5.1 mex and the Grundy value

Definition 5.1 (mex)**.** The *mex* (minimal excludant) of a set S of non-negative integers is the smallest non-negative integer *not* in S: $\operatorname{mex}\{1,2\} = 0$, $\operatorname{mex}\{0,1,3\} = 2$, $\operatorname{mex}\{\} = 0$.

Definition 5.2 (Grundy value)**.** The *Grundy value* (or SG value) $g(p)$ of a position p in a finite impartial game under normal play is

$$g(p) \;=\; \operatorname{mex}\{\, g(q) \;:\; q \text{ is an option of } p \,\},$$

where an *option* is a position reachable in one move and terminal positions have $g = 0$.

The definition mirrors Theorem 2.1 closely enough that one fact falls out immediately: $g(p) = 0$ if and only if p is a P-position. Indeed $g(p) = 0$ says no option has value 0; since options of P-positions are all N-positions and vice versa, the two classifications coincide by the same induction. The extra content of the *number* rather than the label is this lemma, the engine of everything below:

Lemma 5.1 (Reachability of all smaller values)**.** If $g(p) = v$, then for every t with $0 \le t < v$ there is an option q of p with $g(q) = t$; and no option has $g(q) = v$.

Proof. Immediate from the definition: v is the mex of the option values, so $0, 1, \ldots, v-1$ all occur among them and v does not. □

A position with Grundy v therefore *behaves like a Nim pile of size* v: it can be "reduced" to any smaller value (Lemma 5.1) but never to its own value. That sentence is the whole theorem, as the next subsection shows.

5.2 The Sprague–Grundy theorem

Definition 5.3 (Sum of games)**.** Given impartial games $G_1, \ldots, G_k$, their *sum* is played on all components at once: a move consists of choosing one component and making a legal move in it; the player unable to move (every component terminal) loses.

Theorem 5.2 (Sprague–Grundy)**.** For a sum $G_1 + G_2 + \cdots + G_k$ of finite impartial games under normal play,

$$g(G_1 + \cdots + G_k) \;=\; g(G_1) \oplus g(G_2) \oplus \cdots \oplus g(G_k),$$

where $\oplus$ is bitwise XOR. In particular the sum is a P-position exactly when the component Grundy values XOR to zero.

Proof sketch. The structure of Bouton's proof (Theorem 4.1) transfers verbatim, with "pile size" replaced by "component Grundy value" and Lemma 5.1 supplying the moves. Let $s = g(G_1) \oplus \cdots \oplus g(G_k)$. If $s \neq 0$, pick a component G_i whose value $u = g(G_i)$ has the highest set bit of s; by Lemma 5.1 a move inside G_i reaches a component with value $u \oplus s < u$, and the new XOR is $s \oplus u \oplus (u \oplus s) = 0$. If $s = 0$, any move changes one component from u to some option value u'; by Lemma 5.1 $u' \neq u$, so the new XOR is $u \oplus u' \neq 0$. The all-terminal sum has XOR 0 and is a loss for the mover, anchoring the induction over the finite game graph. □

The slogan worth memorizing: *a sum of games is a Nim position whose piles are the component Grundy values.* Deciding, finding winning moves, and even misère-style endgame corrections reduce to Nim bookkeeping on those numbers.

5.3 Computing Grundy tables

One component, one table. The implementation is the recursion of Definition 5.2 with a memo array and a local seen-array for the mex; the *local* array matters, because a shared scratch buffer is silently clobbered by the recursive calls.

```
int memo[N + 1];                          // -1 = not computed

int grundy(int n) {
    if (n == 0) return 0;                 // terminal: no options
    int &g = memo[n];
    if (g != -1) return g;
    bool seen[64] = {false};              // local: recursion-safe
    for (int s : {1, 3, 4})
        if (n >= s) seen[grundy(n - s)] = true;
    int v = 0;
    while (seen[v]) ++v;                  // mex over option values
    return g = v;
}
```

Listing 8: Grundy values of one component: subtraction game with moves $\{1, 3, 4\}$.

Two examples calibrate intuition. For the Bash move set $\{1, 2, \ldots, m\}$ the table is $g(n) = n \bmod (m+1)$—exactly the Bash theorem wearing SG clothes: one pile, so the SG theorem says the position is losing iff $g = 0$, which is Bouton-free Bash. For the sparser move set $\{1, 3, 4\}$, Table 3 shows a richer structure: the zeros are no longer an arithmetic progression, and the table settles into period 7 from $n = 7$ onward.

Table 3: Grundy table for the subtraction game $S = \{1, 3, 4\}$; values repeat with period 7 from $n = 7$ on.

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$g(n)$	0	1	0	1	2	3	2	0	1	0	1	2	3	2

Checking one row by hand cements the machinery: $g(6)$ is the mex of $\{g(5), g(3), g(2)\} = \{3, 1, 0\}$, which is 2; $g(7)$ is the mex of $\{g(6), g(4), g(3)\} = \{2, 2, 1\}$, which is 0—so 7 is a P-position even though it is not a multiple of anything obvious. Positions $0, 2, 7, 9$ are the only zeros in range: sparse, and exactly the values the XOR test will consume.

5.4 Sums of games in practice

A contest problem with several independent components now solves itself: compute g for each component (they may be of *different types*—a pile here, a graph token there), XOR, and answer. The winning move, if required, is located with Lemma 5.1: find a component whose value u satisfies $u \oplus s < u$ (equivalently, the highest set bit of s is set in u), then scan that component's options for one carrying value $u \oplus s$—Lemma 5.1 guarantees the scan succeeds.

```
// components[i] = state of game piece i; grundy(state) is
// implemented per component type (piles, strips, tokens, ...).
long long s = 0;
for (int i = 0; i < k; ++i) s ^= grundy(components[i]);
cout << (s ? "First" : "Second") << "\n";

// Optional: output one winning move when s != 0.
for (int i = 0; i < k && s; ++i) {
    int u = grundy(components[i]);
    if ((u ^ s) < u) {                    // highest bit of s set in u
        int target = u ^ s;
```

```
        for (int opt : optionsOf(components[i]))
            if (grundy(opt) == target) { /* move component i there */ }
    }
}
```

Listing 9: Sprague–Grundy decision over k independent components.

5.5 Moves that split a game

The theorem's fine print—"a move is made in exactly one component"—hides the most common modeling gap: some moves *create* components. In cutting games a move slices a board into two independent pieces; afterwards, players move in either piece, so the option's Grundy value is the XOR of the two parts' values. The mex then runs over these XOR-ed option values. The standard example is worth a full listing because it exercises every idea of the section at once.

Example 5.1 (Cutting a chocolate bar)**.** An $w \times h$ grid bar; a move cuts one piece along a grid line into two pieces. Under normal play the player who cannot move (everything is 1×1... or no legal cut remains) loses. In the classical variant, producing any $1 \times k$ strip *loses immediately*, so optimal play only makes cuts whose two parts are both at least two cells wide.

```
short g[205][205];                      // -1 = not computed

int sgCut(int w, int h) {
    if (w < 3 && h < 3) return 0;       // (2,2),(2,3),(3,2): stuck
    short &r = g[w][h];
    if (r != -1) return r;
    bool seen[256] = {false};
    for (int a = 2; a <= w - 2; ++a)    // vertical cut, both >= 2
        seen[sgCut(a, h) ^ sgCut(w - a, h)] = true;
    for (int b = 2; b <= h - 2; ++b)    // horizontal cut
        seen[sgCut(w, b) ^ sgCut(w, h - b)] = true;
    int v = 0;
    while (seen[v]) ++v;                // mex over split-option values
    return r = v;
}
// The whole position is one bar: mover loses iff sgCut(w, h) == 0.
```

Listing 10: Grundy values for the cutting game of Example 5.1.

The two "trivial" branches repay attention. A bar of width or height 1 is terminal-for-strategy purposes (touching it loses), and the smallest bars where a legal cut exists—$(2, 3)$ and $(3, 2)$, whose only cuts produce a strip—have *no* options at all, so their mex is over the empty set and equals 0. Confirming those bases by hand before trusting the table is exactly the kind of five-minute check that Section 9 begs for.

5.6 Periodicity: stretching tables to 10^{18}

Grundy tables of one-pile games are functions of n, and for the standard families they are *eventually periodic*: the subtraction games with finite move sets are a classical theorem, and in practice the period is small—for $S = \{1, 3, 4\}$, period 7 from $n = 7$ (Table 3); for Bash-style sets, n mod $(m{+}1)$ with no preperiod at all. The contest recipe is mechanical:

```
// g[0..N] computed by Listing lst:sgmemo; find a period p whose
// last two consecutive copies agree, or return 0 if none found.
int findPeriod(const vector<int>& g) {
    int N = (int)g.size() - 1;
    for (int p = 1; 2 * p <= N; ++p) {
        bool ok = true;
```

```
        for (int i = N - 2 * p + 1; i <= N - p; ++i)
            if (g[i] != g[i + p]) { ok = false; break; }
        if (ok) return p;               // g repeats with period p
    }
    return 0;
}
// With preperiod o and period p (found by scanning), the answer for
// huge n is:  n < o ? g[n] : g[o + (n - o) % p].
```

Listing 11: Detecting eventual periodicity of a computed SG prefix.

Honesty about the heuristic's limits belongs in the code review, not the submission: *eventual* periodicity is guaranteed for subtraction games, but the preperiod and period are not bounded by $\max S$ (the set $\{1, 3, 4\}$ already has period $7 > 4+1$), and a pattern confirmed on a short window is a conjecture, not a theorem. Sound practice: compute a generous prefix (a few thousand entries, or as the limits allow), require the repeat to hold over two full copies, and when possible validate against a second window before committing. For games beyond subtraction moves—take-away rules that reference the opponent's last move, for instance—periodicity usually fails, and the table must be computed up to the constraint limit directly or abandoned for a smarter state.

6 Games on Graphs: Cycles and Draws

Everything so far assumed finiteness: the game graph is a DAG, so Theorem 2.1 evaluates it bottom-up and every game ends. Token games on graphs violate that constantly—players slide a token along edges and may return to earlier vertices—and with cycles the two-class theory breaks in an interesting way: with best play the game may never end at all. This section develops the three-class theory (win/lose/draw) and its standard algorithm, *retrograde analysis*, which is a reverse BFS and one of the highest value-per-line algorithms in the contest canon.

6.1 Why cycles break the recursion

Put a token on an edge $\{u, v\}$ of an undirected graph; players alternately move the token along any edge to a neighboring vertex; the player unable to move (a dead end) loses. From vertex u with degree one the mover loses immediately. But decide u and v when they are connected only to each other: the mover from u must move to v, the reply returns to u, and neither player ever benefits from stepping off—if stepping off were losing, the opponent would simply not do it. The win/lose recursion of Theorem 2.1 cannot even start: "u is winning if some move reaches a losing vertex" is now a self-referential loop with no base case, and plain memoized DFS either recurses forever or returns garbage.

The fix is conceptual. With cycles, optimal play defines *three* outcomes from any position: the mover can force a win (**WIN**), the mover can force a loss to be avoided forever but cannot force a win (**DRAW**), or the mover loses against perfect play (**LOSE**). Judges either guarantee one of the classes away or ask explicitly for win/draw/lose; either way, the algorithm below computes all three.

6.2 Retrograde analysis in three rules

The algorithm reasons *backwards* from the positions whose value is already known. Classify every position from the mover's perspective, and process:

Technique 6.1 (Retrograde classification)**.**
1. A position with *no outgoing moves* is **LOSE** (normal play: the mover is stuck). Seed a queue with these.
2. When a position is known to be **LOSE**, every predecessor (a position that can move to it) is **WIN**: the mover takes that edge. Enqueue those predecessors.

3. When a position is known to be **WIN**, decrement a counter of *unresolved* moves in each predecessor; a predecessor whose counter reaches zero—all its moves provably lead to WIN—is **LOSE**. Enqueue it.
4. Positions still unclassified when the queue empties are **DRAW**: the mover can always move somewhere unresolved (and can therefore avoid losing forever), but cannot force a win.

The correctness argument is an induction on the queue order. Every LOSE emitted by rule 3 genuinely is one: all its outgoing edges end in WIN positions, against which the mover loses, by induction. Every WIN emitted by rule 2 genuinely is one: it has an edge into a proven LOSE. Conversely no WIN or LOSE is missed: a genuine LOSE must have all options WIN, and standard induction over optimal-play trees shows the rules propagate exactly that; a genuine WIN must have an option LOSE, which the queue delivers. The residual DRAW class is exactly the positions where the mover can force "not-losing" (some edge never proven LOSE, i.e. into a DRAW or self-sustaining region) but has no edge into a proven LOSE. Total work: every vertex and edge is processed once, $O(V+E)$ memory and time over the *state graph*—the state may be (vertex, extra data), not just a vertex, and all complexity claims are in terms of the state count.

```
// States 0..n-1; moves given as edges u -> v.  val: 0 unknown,
// 1 = WIN (mover forces a win), 2 = LOSE, 3 = DRAW.
vector<vector<int>> radj(n);
vector<int> out(n, 0), val(n, 0);
for (auto [u, v] : edges) { radj[v].push_back(u); ++out[u]; }

queue<int> q;
for (int v = 0; v < n; ++v)
    if (out[v] == 0) { val[v] = 2; q.push(v); }   // stuck: LOSE

while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : radj[u]) if (val[v] == 0) {
        if (val[u] == 2) {                  // predecessor can step
            val[v] = 1; q.push(v);          // into a LOSE: WIN
        } else if (--out[v] == 0) {         // all moves -> WIN:
            val[v] = 2; q.push(v);          // forced LOSE
        }
    }
}
for (int v = 0; v < n; ++v)
    if (val[v] == 0) val[v] = 3;            // survivor: DRAW
```

Listing 12: Retrograde analysis: reverse BFS assigning WIN/LOSE/DRAW to every state.

Three implementation notes prevent the standard bugs. First, the *mover's perspective* rule from Listing 1 is what makes the two propagation directions mirror each other; drawing one reverse edge backwards and getting the polarity wrong is the classic failure. Second, the out-degree counters are consumed by rule 3's decrements, so initialize them from the full edge list before the BFS starts. Third, when the problem promises every position decides (no draws), the survivors list is your assertion—an unexpected DRAW usually means a mis-modeled move, not a mis-typed comparison.

6.3 Where it is used

Retrograde analysis is the engine behind several named classics. *Berzerk*-style problems (a token on a circle of n planets, players have private move sets, monster chases player) are the canonical application: the state graph is (position, player-to-move) with an edge per move option, and the algorithm above runs twice, once per player. Repeated-substring and word-game tasks

(appending a letter, the loser completes a forbidden word) reduce to the same pattern over a trie. On the hard end, *Cat and Mouse*-style problems on general graphs run the same backward sweep but with values attached to pairs of positions and a turn counter, which is the natural "next problem" once Listing 12 feels routine. Note the division of labor with the rest of this guide: DAG state graphs are handled by plain memoization (Section 2.3), small vertex sets combined with "no revisits" rules are bitmask territory (Section 7), and everything cyclic lands here.

7 Bitmask Games and Game DP

Many contest games keep their state in a *set of used or remaining items*: numbers drawn from a pool, cards taken from a table, vertices visited by a token. When the item count is at most about 22, the set fits in one integer as a bitmask, and the whole theory of Sections 2–5 runs over 2^n-sized DP tables. This section packages the standard patterns: win/lose over masks, score games over masks, and the parity tricks that shave a dimension off the state. It deliberately reuses the bit-manipulation toolkit—lowest-bit iteration, popcount, subset bookkeeping—and the companion volume of this series covers those primitives in depth.

7.1 The player-to-move pattern over masks

Fix the convention once and never revisit it: `dp[mask]` stores the answer *for the player about to move, given that exactly the items outside the mask (or inside, by taste) remain.* Under this convention the fundamental recursion is one line and contains no turn parameter:

$$\mathrm{dp}[\mathrm{mask}] = \bigvee_{\text{legal move } m} \neg\, \mathrm{dp}[\text{mask after } m].$$

The mover wins iff *some* move leads the opponent into a losing mask. Two derived quantities usually come free: the current score or sum can be computed from the mask alone (bits set so far), which is why these DPs store 2^n entries and not $2^n\times$ extras; and when the game alternates turns and each turn consumes one item, *whose* turn it is equals the popcount parity of the mask—a one-instruction lookup that replaces a boolean state dimension.

7.2 Win/lose over subsets

Example 7.1 (Draw until you exceed)**.** Numbers $1..M$ lie on the table, each usable by at most one player. Players alternately add one unused number to a shared total (which starts at 0); the player whose addition makes the total reach or exceed T wins immediately. Who wins from the empty table?

The state is the set of used numbers; the running total is derived from it; the recursion is the pattern above with an early-exit test for the winning threshold.

```
short memo[1 << 20];                    // -1 unknown, else 0/1

bool canWin(int mask, int M, int T) {   // mover's perspective
    short &r = memo[mask];
    if (r != -1) return r;
    int total = 0;                      // sum of used numbers:
    for (int i = 0; i < M; ++i)         // derived from the mask
        if (mask >> i & 1) total += i + 1;
    bool win = false;
    for (int i = 0; i < M && !win; ++i)
        if (!(mask >> i & 1)) {
            if (total + i + 1 >= T ||
                !canWin(mask | (1 << i), M, T))
```

```
                win = true;              // instant win, or opponent
            }                            // lands in a losing mask
        return r = win;
    }
    // answer: canWin(0, M, T)
```

Listing 13: Win/lose DP over subsets (Example 7.1); the total is derived, not stored.

Complexity is $O(2^M \cdot M)$ time and $O(2^M)$ bytes of memory—viable for $M \leq 22$ within typical limits, tight at 24 with one-byte cells and aggressive memory settings. The *derived-total* habit matters twice over: it halves the state and, more subtly, keeps the memo *consistent* (two paths reaching the same mask automatically agree on the total, so no key collision is possible).

7.3 Score games over subsets

When the game pays out points instead of ending, the win/lose label is replaced by the *margin*: the best (current player's final score minus opponent's final score) achievable from the mask, again from the mover's perspective. The recursion negates the opponent's optimum, exactly like negamax in Section 8:

$$\mathrm{dp}[\mathrm{mask}] = \max_{i \in \mathrm{mask}} \left(v_i - \mathrm{dp}[\mathrm{mask} \setminus i]\right),$$

with $\mathrm{dp}[\emptyset] = 0$. A positive value means the mover can come out ahead; the value itself often is the answer to the "by how much" question judges ask. Lowest-bit iteration over the remaining set keeps the inner loop tight.

```
long long memo[1 << 20];                 // computed lazily
vector<long long> v;                     // item values

long long bestMargin(unsigned mask) {
    if (mask == 0) return 0;             // nothing left: margin 0
    long long &d = memo[mask];
    if (d != LLONG_MIN) return d;        // sentinel = not computed
    long long best = LLONG_MIN;
    for (unsigned rest = mask; rest; rest &= rest - 1) {
        int i = __builtin_ctz(rest);     // lowest remaining item
        best = max(best, v[i] - bestMargin(mask ^ (1u << i)));
    }
    return d = best;
}
// Initialize memo to LLONG_MIN; watch for states that can never
// occur if the rules restrict which items are legal.
```

Listing 14: Score-margin DP over subsets: each move takes item i, gaining v_i minus the opponent's best reply.

7.4 Visited-set games and parity tricks

A token wanders a graph, may never revisit a vertex, and the player unable to move loses (the *Geography* game). The state is (current vertex, visited mask)—$n \cdot 2^n$ states for $n \leq 20$–22, memoized in a flat array indexed `v * (1 < n) + mask`. The recursion is Theorem 2.1 verbatim: the mover wins iff some unvisited neighbor leads to a losing state. The same skeleton covers "no element reused" board games and removal games with adjacency constraints.

```
char memo[20 << 20];                     // -1 / 0 / 1

bool winsGeog(int v, unsigned mask, const vector<vector<int>>& adj) {
    char &r = memo[v << 20 | mask];      // flat index; n <= 20
    if (r != -1) return r;
```

```
    bool win = false;
    for (int u : adj[v])
        if (!(mask >> u & 1) &&
            !winsGeog(u, mask | (1u << u), adj)) { win = true; break; }
    return r = win;                      // stuck mover (no move) loses
}
```

Listing 15: Geography with $n \le 20$: DP over (vertex, visited mask), player-to-move convention.

The parity trick shines in scoring variants of these games: if the game lasts exactly one item per turn, the popcount of the mask says whose turn occurs at that state, so a single stored value serves both players—halving memory and removing turn parameters from transitions. Combined with the subset-sum derivation of Section 7, most bitmask game DPs compress to exactly two ingredients: the mask and the rules.

7.5 Practical notes

Three engineering habits separate smooth submissions from time-limit deaths. First, prefer `vector<char>` or explicit byte arrays over `map` for memo tables: at 2^{22} states a red-black tree is an order of magnitude slower, and maps only earn their cost when reachable states are sparse. Second, when recursion depth approaches n (it does: one bit per level), watch the stack; a bottom-up loop over masks in increasing popcount order—or plain increasing order, provided every transition only sets bits—replaces recursion mechanically. Third, reuse the bit-manipulation idioms: `mask & (mask - 1)` for iterating items, `__builtin_popcount(mask)` for parity, and `(1 << i)` tests for legality; they are not micro-optimizations here but the natural vocabulary of the transitions. When n exceeds the bitmask budget, the escape hatches are Grundy decomposition (Section 5) if components exist, or the search techniques of Section 8 if the game tree is shallow.

8 Game Tree Search: Minimax and Alpha–Beta

The previous chapters analyzed games *globally*: classify every position once, then answer any query in closed form. Sometimes the global table is unaffordable but the *tree of play from one root* is small—the game lasts ten moves, branching is five, and only one starting position matters. Then the right tool is a *search*: walk the tree of possible plays, propagate values from leaves to root, and prune aggressively. This section develops minimax and its standard refinements—negamax, alpha–beta pruning, transposition tables—at contest scale. The section is also the natural home of *scored* games: whenever a problem asks "by how much does the winner win" on a game too irregular for the subset DPs of Section 7, tree search is the fallback.

8.1 Minimax on explicit trees

Score the game from one player's perspective: leaves of the play tree get exact values (win $+1$ / loss -1, or the final score difference), and internal nodes alternate between the two players:

$$V(\text{max node}) = \max_{c} V(c), \qquad V(\text{min node}) = \min_{c} V(c),$$

where max nodes are positions with player A to move (A maximizes her guaranteed value) and min nodes belong to B. The value of the root is the outcome under optimal play; one child per level reconstructs the optimal line. The formulation is correct by exactly the induction of Theorem 2.1, with "win/lose" upgraded to "numbers". A useful sanity property: for win/lose-only games the minimax value of the root equals $+1$ iff the root is an N-position, which makes tree search and the earlier DPs mutually testable.

A full minimax over a tree with branching factor b and depth d costs $O(b^d)$—fine for b^d up to a few million, hopeless beyond. Two upgrades change the exponent: negation symmetry

(negamax) halves the code, and alpha–beta pruning cuts the effective exponent roughly in half under good move ordering.

8.2 Negamax: one player, one sign

Zero-sum games let both node types share one function. Define every value *from the perspective of the player to move at that node*; a child's value flips sign on the way up, because my opportunity is my opponent's loss:

$$V(s) \;=\; \max_{\text{child } c} \bigl(-V(c)\bigr), \qquad V(\text{leaf}) = \text{score from mover's view.}$$

Negamax is minimax with the min folded into a negation, and it is the formulation every contest implementation should start from—the alpha–beta listing below is negamax plus a two-variable window.

8.3 Alpha–beta pruning

Alpha–beta maintains, along the current search path, a window $[\alpha, \beta]$: α is the best value the mover has already secured, β the best alternative the *opponent* has secured elsewhere. Two rules drive it. If a child's value reaches β, the opponent will never let play reach this node—the remaining children are irrelevant, and the node *cuts off*. If a child's value exceeds α, the mover's guaranteed value improves. The cutoffs preserve the *root* value exactly (they may turn internal values into bounds, which only matters for memoization; see below), and with reasonable move ordering the search visits $O(b^{d/2})$ nodes—the same depth for roughly the square root of the work, or double depth for the same work.

```
const long long NEG = -(1LL << 60);     // half-range sentinels:
const long long POS =  (1LL << 60);     // safe to negate

// value(s) and children(s) are game-specific; values from the
// perspective of the player to move at s.
long long negamax(const State& s, long long alpha, long long beta) {
    if (isTerminal(s)) return value(s);

    for (const State& c : children(s)) {   // best moves first!
        long long v = -negamax(c, -beta, -alpha);
        if (v >= beta) return beta;        // cutoff: opponent has
        if (v >  alpha) alpha = v;         // better options elsewhere
    }
    return alpha;                          // our best guarantee
}
// root call:  negamax(root, NEG, POS);
```

Listing 16: Negamax with alpha–beta pruning. Note the half-range sentinels: negating `LLONG_MIN` is undefined behavior.

The single highest-leverage implementation detail is *move ordering*: cutoffs fire early when strong children come first. In contest games the ordering heuristic is nearly always "moves that end the game or immediately win, then captures/large gains, then the rest", plus any domain-specific rule (center squares, high cards). A cheap two-pass scheme—search shallow to rank moves, search deep in ranked order—recovers much of the perfect-ordering bound when no obvious heuristic exists.

8.4 Transposition tables

Different move orders often reach the *same* position; a *transposition table* memoizes node values by canonical state, converting the tree into a DAG search. Two cautions keep it correct. First, alpha–beta with cutoffs may return a *bound* rather than an exact value (a $\geq \beta$ cutoff proves "at least β"); storing bounds as if exact can poison later lookups. The safe contest policy: store a value only when it was computed with the window not cutting off, or store exact/bound flags—or skip memoization entirely when the state count is small enough for plain recomputation. Second, the state key must be *canonical*: normalize symmetries (sorted piles, canonical board rotation) before hashing, or identical positions will duplicate. In practice, an `unordered_map<State, long long>` with a decent hash handles the state counts that make search necessary at all; if the count exceeds memory, the game belongs to Section 7 or Section 5 instead.

8.5 Choosing the tool

A closing triage, since this chapter is the "when nothing else fits" branch of the guide. If states decompose into independent components, use Grundy theory (Section 5); if the state is a set of at most ∼20 items, use the subset DPs (Section 7); if the state graph is small and cyclic, use retrograde analysis (Section 6); if the game tree from a single root is shallow—tens of millions of nodes after pruning—use this chapter. Typical guests of this branch: scored removal games with exotic legality rules, board games on tiny grids, and any task whose constraints ("at most 15 moves") exist only to make the tree finite. The techniques compose: a search whose states are bitmasks, memoized by popcount-canonical keys, is a standard hybrid in editorials.

9 Pitfall Compendium

Game-theory code fails in characteristic ways: the theory is short, so the bugs are short too, and they survive testing because small positions hide them. This section collects the traps in one place for a five-minute pre-contest re-read, in the same spirit as the rest of this series. Each box names its failure class; the closing paragraph names the meta-habit that catches all of them.

P is for Previous, N is for Next

A P-position is a win for the *previous* player—that is, a loss for the player *to move*. Misreading the label flips every answer and every proof. The safe habit, used throughout this guide: never reason about "who" in the abstract; every DP and every theorem statement here is keyed to *the player about to move from the position*, and answers are translated to player names only at the output boundary ("first/second player wins").

The empty position and zero piles

The XOR of *no* components is 0, and the empty game is a loss for the mover: a request like "all piles zero" or "no numbers left" must answer *second player wins*, never "first wins" by a vacuous formula. The same edge lurks in the misère branch: zero single-stone piles means the mover is stuck and loses, even though zero is even. Every formula in Table 2 should be audited for its empty-input behavior before submission.

Misère: check the all-ones branch first

The misère Nim rule (Theorem 4.2) is a *conditional*: if every pile ≤ 1, use the parity of the ones count; *otherwise* use the nim-sum. Applying the nim-sum unconditionally loses the endgame—$(1, 1)$ has nim-sum zero but the mover *wins* under misère play. The branch order is the whole algorithm; sort it out before writing any code.

mex runs over option values, not move sizes

$\mathrm{mex}\{1,2,3\}$ is 0, and mex of the *empty set* is 0. Two corollaries bite in practice: the Grundy recursion marks options by *their Grundy values* (Listing 8), never by the sizes of moves; and a move that *splits* a component contributes the XOR of the parts first (Listing 10), so the seen-array must be indexed by those combined values. Forgetting the split-XOR is the single most common Grundy bug in submitted code.

The SG theorem needs genuinely independent parts

Sprague–Grundy applies when a move touches *exactly one* component and components share no resources. Games with a shared budget (a global move counter, one shared hand limit, total-per-turn caps) or interaction rules (a move in one component disables a move in another) are *not* sums, and XOR-ing their Grundy values is garbage. The repair is to enlarge the state until the interaction is inside one game—or to recognize the special structure (several interaction rules are staircase or Moore-style transformations in disguise).

Cycles plus memoized DFS equals wrong answers

On cyclic state graphs, plain `wins(state)` recursion neither terminates nor fails loudly—it returns values that look plausible and are meaningless. Any problem whose moves can return to earlier states (graph tokens, undirected edges, "pass"-like options) demands retrograde analysis (Listing 12) and a three-valued answer space. Check the story for loops before committing to a DAG recursion; "players may move back" is the smell.

Zero moves and off-by-one move sets

A subtraction set containing 0 makes the game infinite—a "move" that changes nothing loops forever, and DP tables silently index $g(v-0)$ as $g(v)$, a self-referential cycle. Likewise the Bash constant is $m+1$, not m (the optimal round consumes $m+1$ stones in total), and "at most k piles per turn" is Moore's $\equiv 0 \pmod{k+1}$, not $\pmod{k}$. When transcribing a statement into a move set, write the set explicitly and check its extremes first: 0, 1, the cap, and the cap plus one.

Golden ratios and floating-point floors

Wythoff's cold positions (Listing 7) hinge on $\lfloor d\varphi \rfloor$ for potentially large d. In `double`, the error in $d\varphi$ reaches order 1 once d approaches 10^{15}, and the floor snaps to the wrong integer. Use `long double` (64-bit mantissa on x86), keep inputs within the verified range, and when in doubt test the neighbors $\lfloor d\varphi \rfloor$ and $\lfloor d\varphi \rfloor \pm 1$ against the defining properties of the cold pairs; or implement the floor in integer arithmetic from a high-precision rational approximation of φ.

Read the losing condition literally

"The player unable to move loses", "the player who takes the last stone loses", and "the player who takes the last stone *after the pile is empty* loses" are three different games—the middle one is misère, the first is normal play, and the third changes the terminal class of positions that still have moves. Similarly, "each player may move in at most k piles" upgrades Nim to Moore's Nim_k, whose P-positions are *not* the nim-sum test. Restate the ending condition and the move restrictions as a bulleted list in the margin before coding; misread endings are the most expensive bugs in this topic because they pass every small symmetric test.

Memo tables: memory, depth, and sparsity

A 2^{22}-entry `char` table is 4 megabytes and fine; 2^{26} is 64 megabytes and often fatal; an `unordered_map` over the same states is slower by an order of magnitude and allocates unpredictably. Choose the table from the state budget: dense arrays when states are near-dense, maps only when genuinely sparse. Watch recursion depth too—one level per item means depth n–22 is safe, but graph games and cutting games can recurse by dimensions; convert to iterative or bottom-up order when the depth approaches the stack. Finally, sentinels must be unreachable values (`LLONG_MIN` is *not*, since it negates to itself—Listing 16 uses half-range sentinels for exactly this reason).

A closing meta-pitfall, because it is the one that survives code review: *testing only on showcase positions*. Game formulas agree with the brute-force DP on the small symmetric cases and diverge exactly at the edges—empty positions, all-ones endgames, single big piles, and the largest allowed input. The reliable test harness has four members: the empty position, the smallest non-terminal position, the misère endgame if the problem is misère, and a randomized simulator that plays the *formula* against a brute-force optimal player over thousands of small positions. If the formula's choices beat or tie the brute force everywhere and agree on the decision at every position, the formula is right with a confidence no amount of staring at the proof provides; if they disagree once, you have found your bug before the judge did. Ten minutes of harness beats an hour of resubmitting.

10 Cheat Sheet and Practice Roadmap

10.1 The one-page cheat sheet

Table 4 compresses the guide into the tests and formulas you will actually apply. Read the "Meaning" column as sight-reading practice in both directions: given the position, name the test; given the test, reconstruct the position class it decides. Every row is proved by the two clauses of Theorem 2.1—when a row's proof is needed under pressure, replay those two clauses.

Table 4: Game-theory quick reference (normal play unless noted; "mover" = the player to move).

Test / formula	Meaning	Notes / source
no legal moves	mover loses	normal-play base case
win iff ∃ move to P; P iff all moves win	fundamental recursion	Theorem 2.1
$n \equiv 0 \pmod{m+1}$	Bash: mover loses	take 1..m; answer in $O(1)$
$\bigoplus_i p_i = 0$	Nim: mover loses	Bouton's theorem
pile p with top bit of s; play $p \oplus s$	Nim winning move	$s \neq 0$; one pass
all piles ≤ 1: ones-count parity; else nim-sum	misère Nim decision	check branch *first*
$a_1 \oplus a_3 \oplus a_5 \oplus \cdots = 0$	staircase: mover loses	pairing argument
every column sum $\equiv 0 \pmod{k+1}$	Moore's Nim_k: mover loses	$\leq k$ piles per turn
n is a Fibonacci number	Fibonacci Nim: mover loses	winning take: largest Zeckendorf part
$(a, b) = (\lfloor n\varphi \rfloor, \lfloor n\varphi \rfloor + n)$	Wythoff cold pair	`long double` caution
$g(p) = \mathrm{mex}\{g(q) : q \text{ option}\}$	Grundy value	$g = 0$ iff P-position

Test / formula	Meaning	Notes / source
$\bigoplus_i g(G_i)$	SG decision over a sum	0 means mover loses
split move: $g_1 \oplus g_2$ before mex	cutting games	Listing 10
SG prefix, find p with two equal copies	eventual periodicity	stretch to 10^{18}
outdeg 0: LOSE; sees LOSE: WIN; all see WIN: LOSE; else DRAW	retrograde rules	$O(V+E)$ reverse BFS
dp[mask] = $\bigvee_m \neg$ dp[mask after m]	bitmask win/lose	$O(n \cdot 2^n)$
dp[mask] = $\max_i(v_i - \text{dp}[\text{mask} \setminus i])$	score margin over subsets	0 at the empty mask
popcount(mask) mod 2	whose turn it is	drops a state dimension
sum from mask, not from state	derived totals	keeps memo keys canonical
$-V$(child)	negamax	values from mover's view
cutoff when $v \geq \beta$	alpha–beta	$O(b^{d/2})$ with ordering
memo keyed by canonical state	transposition table	bounds are not exact values
complement to m+1 each round	Bash strategy	pairing template
mirror under an involution	copycat: second player wins	symmetry check
eat one cell, steal the reply	strategy stealing	first player wins (Chomp)
coloring / parity of the resource	invariant arguments	one-unit move games

10.2 Practice roadmap

Techniques stick when each is exercised against real judge problems within a day or two of reading. The list below maps this guide's sections to well-known problems, ordered so that each row assumes only the rows above it; all are reachable from the standard judges.

- **Foundations and the recipe** (Sections 2–3): LeetCode *Nim Game* (292), *Divisor Game* (1025), *Remove Colored Pieces if Both Neighbors Are the Same* (2038); any "coins in a line" task. Goal: state–moves–base– budget written out before any code, and both clauses of the proof checked for the pattern you conjecture.
- **Nim and variants** (Section 4): CSES *Nim Game I* and *Nim Game II*; LeetCode *Game of Nim* (1908); POJ *Matches Game* (2234) for the classic; Codeforces *Sequential Nim* (1382B) for endgame reasoning; POJ *Stone Game* (1067) for Wythoff. Goal: winning-move extraction and the misère branch from memory, one pass each.
- **Grundy and sums** (Section 5): LeetCode *Stone Game IV* (1510); POJ *Cutting Game* (2311) for split moves; Codeforces *Interesting Game* (87C); tasks tagged *games* on any judge whose editorial says "Grundy". Goal: a full SG solver with split-move handling, plus the periodicity recipe validated on a second window.
- **Graph games** (Section 6): Codeforces *Berzerk* (786A)—retrograde analysis twice; Codeforces *MADMAX* (917B)—DAG game DP with a letter state; LeetCode *Cat and Mouse* (913) as the hard end. Goal: Listing 12 reproduced from scratch, polarity and all.

- **Bitmask game DP** (Section 7): LeetCode *Can I Win* (464); *Predict the Winner* (486) and *Stone Game* (877) as score games (interval or mask); Geography-style tasks with $n \leq 20$. Goal: the player-to-move mask idiom, derived totals, and popcount parity applied without notes.
- **Tree search** (Section 8): any tic-tac-toe-family solver; minimax tasks on judges that tag them ("alpha-beta", "minimax"); score games too irregular for Section 7 but with shallow trees. Goal: negamax with alpha–beta and a transposition table, sentinel-safe.

10.3 Closing advice

Three habits separate contestants who "know" this material from those who solve with it. First, restate the ending condition literally before anything else—normal or misère, stuck-mover or last-taker, one pile per turn or k—because every formula in Table 4 is conditional on those sentences, and misreadings pass every small symmetric test. Second, classify the problem before coding, in this order: formula shelf (Section 4), Grundy decomposition (Section 5), subset DP (Section 7), retrograde BFS (Section 6), tree search (Section 8); the constraint sizes usually point at exactly one branch. Third, keep the simulator harness of Section 9 as standing equipment—a brute-force optimal player over small positions catches in one minute what would otherwise cost three submissions. The results in this guide are a century old, stable, and universal, which is precisely why they are worth knowing perfectly rather than approximately: Bouton's theorem does not change between contests, and neither does the value of proving your pattern before trusting it.

Where to go next. For reference material beyond this guide, the *CP-Algorithms* website (cp-algorithms.com) has rigorous articles on Nim and the Sprague–Grundy theorem and on games on arbitrary graphs, each with proofs and tested code. Thomas Ferguson's lecture notes *Game Theory* (UCLA, freely available) develop everything in Sections 2 and 5 with more care and continue into partisan games. For the deep end—partisan theory, cold positions, and the surreal numbers hiding behind them—Berlekamp, Conway, and Guy's *Winning Ways for Your Mathematical Plays* remains the definitive and remarkably playful source. And for the bitmask machinery that Section 7 leans on, the companion volume of this series, *Bit Manipulation for Competitive Programming*, covers the primitives in full. Reading one chapter of any of these per week, alongside the roadmap above, moves every technique here from "familiar" to "reflex" within a season.